DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA0607 Design and Analysis of Algorithms

Assignment 1 - CO1

Strategy for a Large-Scale Student Record System

Linear search | Recursive binary search with merge sort | Hash table with chaining

https://github.com/192571027/CSA0607-

Name	RITHIKA G
Register number	____192571027_____
Course	CSA06 - Design and Analysis of Algorithms
Course outcome	CO1 - algorithm efficiency, asymptotic analysis, recurrences
Submission date	11 October 2026

Executive Summary

This report designs, analyses, implements and experimentally validates three different strategies for finding a registration number in a university student-record database that grows from 1,000 to 1,000,000 records, and then re-evaluates the design for a workload of 50,000 searches per hour with one batch of new records per day.

Table S.1 - Results at a glance

	A Linear search	B Sort + recursive binary search	C Hash table (chaining)
Strategy	Brute force, iterative	Divide-and-conquer sort, decrease-and-conquer search, recursive	Hashing (space-time trade-off), iterative
Preprocessing	none	Theta(n log n)	Theta(n)
Search (worst / average)	Theta(n)	Theta(log n)	Theta(n) worst / Theta(1) expected
Recurrence	-	C(n) = C(floor(n/2)) + 1 -> floor(log2 n) + 1	-
Mean search, n = 1,000,000 (measured)	10,789 us	3.17 us	0.54 us
Preprocessing, n = 1,000,000 (measured)	0	2.53 s	1.13 s
Extra memory, n = 1,000,000	0	8 MB	97 MB

Final recommendation (original problem): sorted array with binary search - deterministic worst case of 20 comparisons for a million records, small memory, supports ordered queries, preprocessing repaid after a few hundred searches. **Decision (50,000 searches/hour, daily updates): MODIFY** - keep the sorted master copy but serve exact-match lookups from a hash index that is extended in the nightly batch: about 1.6 us mean latency in the 50,000-request test, 2.5x lower than binary search, with a nightly insertion cost of milliseconds. Linear search is rejected: it needs about 12 minutes of CPU per hour and 4 CPU-hours per day for the same traffic.

> **Note.** Experimental numbers were produced by the code supplied in the repository on one machine (Python 3.12, single thread). Absolute times will differ on another computer; counts of basic operations and growth rates are machine independent. Re-run the scripts and, if required, replace the rendered output images with screenshots from your own run.

Contents

Task 1 - Algorithm Design

Task 2 - Identify the Basic Operation

Task 3 - Mathematical Time-Complexity Derivation

Task 4 - Best, Average and Worst-Case Analysis

Task 5 - Asymptotic Analysis

Task 6 - Recurrence Relation Formulation

Task 7 - Solving the Recurrence Relations

Task 8 - Implementation and Testing

Task 9 - Experimental Performance

Task 10 - Theory vs Experiment

Task 11 - Final Design Recommendation

Task 12 - Design Challenge

Conclusion

Appendix A - Repository structure and README summary

Appendix B - Source code listings

Task 1 - Algorithm Design

The problem is **membership search with retrieval**: given a registration number (a 9-digit integer key) decide whether it exists among n student records and return its position. Three algorithms from three different design paradigms were selected. Iterative and recursive forms of the same idea are deliberately **not** counted as different approaches.

Table 1.1 - The three approaches

#	Algorithm	Design strategy	Style	Preprocessing	Why it is investigated
A	Linear (sequential) search	Brute force	Iterative	None	Baseline. Works on the raw, unsorted table; the cost every smarter design must beat.
B	Binary search on a sorted array	Decrease-and-conquer (problem halves each step); preprocessing by divide-and-conquer (merge sort)	Recursive	Merge sort, O(n log n)	Exploits ordering of keys. Shows the trade-off 'pay once to sort, then search very cheaply'.
C	Hash table with separate chaining	Space-time trade-off (input enhancement by hashing)	Iterative (chain walk)	Build table, O(n)	Gives (expected) constant-time lookup; shows what extra memory can buy.

Approach A - Linear search (iterative, brute force)

Idea. Examine records one after another from the first position until the key is found or the table ends. No assumption about order is needed, therefore no preprocessing.

```
ALGORITHM LinearSearch(A[0..n-1], key)
    for i <- 0 to n-1 do
        if A[i] = key then          // basic operation
            return i
    return -1                       // not found
```

Justification. It is the simplest correct method, it is the natural benchmark, and for tiny databases or one-off queries its zero preprocessing may actually be optimal.

Approach B - Recursive binary search on a sorted array

Idea. If the registration numbers are kept in sorted order, compare the key with the middle element. Equality ends the search; otherwise the key can only lie in one half, so the other half is discarded and the same procedure is applied recursively to the remaining half. **Preprocessing:** the records must first be sorted; merge sort (divide-and-conquer, guaranteed O(n log n)) is used, and its cost is included in every analysis.

```
ALGORITHM BinarySearch(A[lo..hi], key)          // A is sorted
    if lo > hi then return -1                   // base case: empty range
    mid <- floor((lo + hi) / 2)
    if A[mid] = key then return mid             // basic operation: key vs A[mid]
```

```
  if key < A[mid] then
    return BinarySearch(A, key, lo, mid-1)    // recurse on left half
  else
    return BinarySearch(A, key, mid+1, hi)    // recurse on right half


ALGORITHM MergeSort(A[0..n-1])            // preprocessing
  if n <= 1 then return A
  L <- MergeSort(A[0..n/2-1]);  Rr <- MergeSort(A[n/2..n-1])
  return Merge(L, Rr)                     // linear merge of two sorted lists
```

Justification. It is the classic example of a recursive algorithm whose recurrence can be formulated and solved (Tasks 6-7), and it represents the 'sort once, search many times' strategy.

Approach C - Hash table with separate chaining

Idea. A hash function h(key) maps a registration number directly to one of m buckets (m is a prime near n/0.75, so the load factor is alpha = n/m = 0.75). Every bucket holds a short chain of record indices. Searching computes h(key) and scans only that one chain. **Preprocessing:** every record is inserted once into its bucket. The hash function used is Knuth's multiplicative hash, h(k) = ((k x 2654435761) mod 2^32) mod m, which spreads even sequential registration numbers evenly.

```
ALGORITHM BuildHash(A[0..n-1])                // preprocessing
  m <- next_prime(n / 0.75);  T[0..m-1] <- empty
  for i <- 0 to n-1 do
    append i to chain T[ h(A[i]) ]


ALGORITHM HashSearch(T, A, key)
  chain <- T[ h(key) ]
  for each index idx in chain do
    if A[idx] = key then          // basic operation
      return idx
  return -1
```

Justification. It breaks the comparison-based lower bound of Omega(log n) for searching by using the key's *value* rather than only comparing keys, and it represents the memory-for-speed strategy.

Note. Return value: all three algorithms return a position (index) for a found key and -1 otherwise. In a production system each slot would additionally hold a pointer to the full student record; this changes memory use by a constant factor only.

Task 2 - Identify the Basic Operation

The basic operation is the operation executed most often in the innermost part of the algorithm, whose count grows with n and therefore dominates the running time. Every other statement (index arithmetic, assignments, function-call overhead) is executed at most a constant number of times per basic operation, so counting basic operations measures the algorithm's growth rate independently of hardware.

Table 2.1 - Basic operations

Algorithm	Basic operation	Where it occurs	Why it is appropriate	How its count changes with n
A - Linear search	Comparison A[i] = key	Body of the single for-loop; once per examined record	It is the only work inside the loop; loop control is proportional to it. Each execution examines exactly one record.	Between 1 and n. It grows in direct proportion to n for failed searches (always n) and on average for successful ones ((n+1)/2).
B - Binary search	Three-way comparison of key with A[mid]	Once per recursive call (once per level of the recursion)	Each call performs exactly one such comparison and then discards half of the range, so the number of calls equals the number of comparisons.	Equals the number of times n can be halved: at most floor(log2 n)+1. Multiplying n by 1,000 adds only about 10 comparisons.
B - Preprocessing (merge sort)	Comparison of two elements in Merge	Inner while-loop of Merge	Each comparison moves one element to the output, so total work is proportional to comparisons.	About n log2 n - n comparisons for random input.
C - Hash table	Comparison A[idx] = key inside the chain (the hash computation is one O(1) step per search)	For-loop over the chain of bucket h(key)	Hashing costs the same for every n, so the part that can vary is the chain scan; its length is the only n-dependent cost.	Expected chain length is alpha = n/m (about 0.75 here), independent of n if m grows with n. So it stays about 1-2 comparisons for any n.
C - Preprocessing (build)	Hash computation + append, once per record	Loop of BuildHash	Exactly n executions, each O(1).	Exactly n.

Relation to behaviour. In A the operation count is governed by *where the key is*, in B by *how many times the range can be halved* (a property of n only), in C by *how crowded the key's bucket is* (a property of the load factor, not of n). These three different dependencies are exactly what produce the linear, logarithmic and constant growth derived in Task 3 and confirmed by counters in Task 9.

Task 3 - Mathematical Time-Complexity Derivation

Notation: **n** = number of records (input size); **C(n)** = number of basic operations; **T(n) = c x C(n) + c0**, where c is the constant cost of one basic operation. **q** = number of searches performed after preprocessing.

A - Linear search

If the key is at position i (1-based) the loop executes the comparison exactly i times; if it is absent the loop runs to the end, n times.

```
C_best = 1     C_worst = n     C_unsuccessful = n
```

Average case for a successful search, assuming each of the n positions is equally likely (probability 1/n):

```
C_avg(success) = sum_{i=1..n} i * (1/n) = (1/n) * n(n+1)/2 = (n+1)/2
```

If a fraction p of queries is successful and (1-p) unsuccessful:

```
C_avg = p(n+1)/2 + (1-p)n = n(1 - p/2) + p/2
```

Since T(n) = c*C(n)+c0 and C(n) is a linear function of n in every case, T(n) belongs to Theta(n) (worst and unsuccessful case). Preprocessing = 0. Cost of q searches: **q x n** (worst case) or about **q x n/2** (successful average).

B - Binary search + merge sort

Search. Each call performs one comparison and passes a sub-array of at most floor(n/2) elements to the next call, so C(n) = C(floor(n/2)) + 1 with C(0)=0, C(1)=1. Unrolling for n = 2^k: C(n) = k + 1 = log2 n + 1; the exact worst-case count for any n is C_worst(n) = floor(log2 n) + 1 (proved in Task 7). Best case C_best = 1 (key at the middle position).

Average (successful) for n = 2^k - 1. In the complete decision tree there are 2^(i-1) elements found after exactly i comparisons (levels i = 1..k). Therefore

```
C_avg = (1/n) * sum_{i=1..k} i * 2^(i-1) = ((k-1) * 2^k + 1) / n = ((n+1)/n) * log2(n+1) - 1
```

For n = 1,000,000 this is 19.93 - 1 = 18.93 comparisons - so the average is only about one comparison smaller than the worst case, because half of all elements sit on the bottom level. For an unsuccessful search every one of the n+1 gaps is reached after about log2(n+1) comparisons.

Preprocessing (merge sort). The merge step compares the heads of two sorted lists and outputs one element per comparison, so merging lists of total size m takes at most m - 1 comparisons (at least m/2). Hence W(n) = 2W(n/2) + n - 1, W(1)=0, which solves (Task 7) to

```
W(n) = n log2 n - n + 1      so merge sort is Theta(n log n) in every case; best case (n/2) log2 n
```

Overall for q searches:

```
T_B(n, q) = [ n log2 n - n + 1 ]  +  q * [ floor(log2 n) + 1 ]   =   Theta(n log n + q log n)
```

C - Hash table (separate chaining)

With m buckets and n keys, load factor alpha = n/m. Under the *simple uniform hashing* assumption (each key equally likely to fall into any bucket, independently) the expected length of a chain is alpha.

Unsuccessful search scans the whole chain: C = alpha. **Successful search:** the key that was inserted i-th has, on average, (i-1)/m keys ahead of it in its chain, so the average over all keys is

```
C_avg(success) = (1/n) * sum_{i=1..n} [ 1 + (i-1)/m ] = 1 + (n-1)/(2m) = 1 + alpha/2 - alpha/(2n)  ~  1 + alpha/2
```

Both are independent of n when m grows proportionally to n (alpha constant = 0.75), so T(n) = c*(1+alpha/2)+c_hash = Theta(1) expected. Worst case: every key lands in one bucket and the chain scan becomes linear, C_worst = n, T = Theta(n) (extremely unlikely with a good hash function).

Preprocessing. Allocating m = Theta(n) slots and inserting n keys with O(1) work each gives Theta(n).
Overall for q searches:

```
T_C(n, q) = n + q * (1 + alpha/2)   =   Theta(n + q)   (expected)
```

Combined comparison of preprocessing + searching

Table 3.1 - Complexity functions

Algorithm	Preprocessing	One search (worst / avg)	Total for q searches
A Linear	0	n / (n+1)/2	Theta(q n)
B Sort + binary	Theta(n log n)	floor(log2 n)+1 / ~log2 n - 1	Theta(n log n + q log n)
C Hash	Theta(n)	n / 1 + alpha/2 (expected)	Theta(n + q) expected

Worked example (n = 1,000,000; q = 50,000 searches, 90% successful): A needs about 2.76e+10 comparisons; B needs about 1.96e+07 (of which 18,674,355 are the one-off sort); C needs about 1.07e+06 (of which 1,000,000 are the one-off build). Counting comparisons only, the sort has paid for itself after about n log2 n / (n/2) = 2 log2 n = 40 searches; in measured time (Table 9.4) the break-even is a few hundred searches because one comparison inside the Python sort loop costs more than one inside the tight linear-search loop.

Task 4 - Best, Average and Worst-Case Analysis

A - Linear search

- **Best case - the key is the first record.** The first comparison succeeds: C = 1, T = Theta(1). It occurs because the search order is fixed from the front of the table; no property of n matters.
- **Average case - the key is equally likely at any position.** On average half of the table is scanned before the key is met: C = (n+1)/2. A successful search is therefore only about twice as fast as a failed one, but both are Theta(n).
- **Worst case - the key is the last record or absent.** Nothing can be concluded about absence until all n records have been inspected: C = n. Absent keys are the *typical* worst case in a registration system (typos, students who never enrolled).

B - Binary search (on the sorted array)

- **Best case - the key is at the middle index.** It is found in the first call, C = 1, Theta(1).
- **Average case.** Half the elements are on the deepest level of the implicit decision tree and a quarter on the next, so the average successful search needs ((n+1)/n) log2(n+1) - 1 comparisons, i.e. about **log2 n - 1**; Theta(log n).
- **Worst case - the key is absent or located on the deepest level.** The range must be halved until it is empty: C = floor(log2 n) + 1. The difference between best, average and worst arises only from *when* the middle probe happens to hit the key; the range shrinks by half in every case, which is why even the worst case is logarithmic and the data order (random, nearly sorted) does not matter after sorting.
- **Preprocessing (merge sort).** Best, average and worst are all Theta(n log n): the number of levels (log n) is fixed and each level does about n work; only the constant changes (best (n/2) log n if the halves are already ordered, worst n log n - n + 1).

C - Hash table

- **Best case - the key sits alone (first) in its bucket**, C = 1; for an absent key whose bucket is empty, C = 0 (only the hash is computed).
- **Average case - keys are spread uniformly.** The chain has expected length alpha (0.75): C = 1 + alpha/2 for successful and alpha for unsuccessful searches, independent of n. Theta(1).
- **Worst case - all (or very many) keys collide into the same bucket.** The structure degenerates to one linked list and C = n, Theta(n). It occurs for a poor hash function (e.g. k mod m with systematically structured registration numbers) or deliberately chosen keys. The multiplicative hash and the measured maximum chain length (at most 6 for any tested n) show that this does not happen in practice here.

Table 4.1 - Case summary and measured counts

Algorithm	Best	Average (successful)	Worst	Measured at n = 1,000,000 (best / avg / worst)
A Linear	1	(n+1)/2	n	1 / 501,371 / 1,000,000
B Binary	1	~log2(n+1) - 1	floor(log2 n)+1	1 / 18.93 / 20
C Hash	1 (0 if empty bucket)	1 + alpha/2	n	1 / 1.40 / 5 (max chain observed)

Task 5 - Asymptotic Analysis

Table 5.1 - Asymptotic classification

Algorithm / quantity	O (upper)	Omega (lower)	Theta (tight)	Comment
A Linear search, any input	O(n)	Omega(1)	none for 'all inputs'	Best case Theta(1), worst case Theta(n) - no single Theta covers every input.
A Linear, failed search / worst case	O(n)	Omega(n)	Theta(n)	Every record is inspected.
B Binary search, any input	O(log n)	Omega(1)	none for 'all inputs'	Worst case and failed search: Theta(log n).
B Merge sort (preprocessing)	O(n log n)	Omega(n log n)	Theta(n log n)	Same in best, average, worst; comparison sorting cannot beat n log n.
C Hash search, expected	O(1)	Omega(1)	Theta(1) expected	Worst case O(n) if all keys collide.
C Hash build	O(n)	Omega(n)	Theta(n)	Every record must be touched.

Ordering of growth rates. 1 << log n << n << n log n. For example, lim (log2 n)/n = 0 (by L'Hopital, (1/(n ln 2))/1 -> 0), so log n grows strictly slower than n, and n/log2 n -> infinity: the advantage of B over A *grows without bound* as n grows. Likewise the advantage of C over B grows like log n.

Meaning for the university database (1,000 to 1,000,000+ records)

Table 5.2 - Operation counts at the required sizes

n	A Linear (avg / worst)	B Binary (worst)	C Hash (expected)	Factor n grows	A grows by	B grows by	C grows by
1,000	500 / 1,000	10	~1.4	-	-	-	-
10,000	5,000 / 10,000	14	~1.4	10x	10x	1.4x	1x
100,000	50,000 / 100,000	17	~1.4	10x	10x	1.2x	1x
1,000,000	500,000 / 1,000,000	20	~1.4	10x	10x	1.2x	1x

- When the database grows 1,000-fold, linear search does **1,000 times more work per query**; its cost increase is the same as the data increase, so it eventually becomes impractical even if it is acceptable at 1,000 records.
- Binary search does only **twice** as much work (10 to 20 comparisons) - the 1,000-fold growth costs about 10 extra comparisons. The price is the Theta(n log n) sort, paid once and again whenever the data changes.
- The hash table's per-query work does not change at all as n grows (provided the table is sized with n). Its price is Theta(n) memory for the table and the loss of key order.
- Because O-notation hides constants, the asymptotic ranking says nothing about small n (at n = 1,000 the three are all microseconds) - this is why Tasks 9-10 measure real behaviour, and why preprocessing (a constant-factor/lower-order cost per search when q is large) must be examined together with the number of searches q.

Task 6 - Recurrence Relation Formulation

The recursive approach is **recursive binary search** (Approach B). The recurrence is derived directly from the code in Section 1.

Table 6.1 - Components of the recurrence

Component	Value for BinarySearch(A, key, lo, hi)	Reason (from the algorithm)
Input size of the original problem	n = hi - lo + 1 (number of elements still under consideration)	The call only ever looks at A[lo..hi].
Number of recursive calls	a = 1	After comparing with A[mid], the code takes exactly one of the two branches (left or right); the other half is never examined.
Size of the sub-problem	left: mid - lo = floor((n-1)/2); right: hi - mid = floor(n/2). Worst case (larger half): floor(n/2), i.e. n/b with b = 2	mid is the centre of the range.
Work outside the recursive call	f(n) = 1 basic operation (one comparison key : A[mid]); computing mid and the range tests are O(1)	No loop is executed in one call.
Base cases	n = 0 (lo > hi): C(0) = 0 comparisons, returns -1. n = 1: C(1) = 1 comparison	Empty range ends the recursion.

Combining the components (counting comparisons in the worst case):

$$C(n) = C(\text{floor}(n/2)) + 1 \quad \text{for } n >= 2, \qquad C(1) = 1, \quad C(0) = 0$$

In terms of running time $T(n) = T(n/2) + \text{Theta}(1)$, $T(1) = \text{Theta}(1)$.

Recurrence of the preprocessing step (merge sort)

MergeSort makes **two** recursive calls on halves of size n/2 and then performs a linear merge costing at most n - 1 comparisons:

$$W(n) = 2\,W(n/2) + (n - 1) \quad \text{for } n >= 2, \qquad W(1) = 0$$

Because the sort is a part of the total cost of approach B, its recurrence is solved in Task 7 as well.

Task 7 - Solving the Recurrence Relations

7.1 Binary search: C(n) = C(floor(n/2)) + 1, C(1) = 1

Method selection. The recurrence has the standard divide-and-conquer form T(n) = a T(n/b) + f(n) with a = 1, b = 2, f(n) = 1, so both the *Master Theorem* and the *backward-substitution (iteration)* method apply. The iteration method is shown first because it exposes the structure; the Master Theorem confirms it; induction (substitution method) proves the exact floor/ceiling form.

(a) Backward substitution, n = 2^k

```
C(n) = C(n/2) + 1
   = [C(n/4) + 1] + 1 = C(n/4) + 2
   = C(n/8) + 3
   ...
   = C(n/2^i) + i
```

The recursion stops when n/2^i = 1, i.e. i = k = log2 n. Then C(n) = C(1) + log2 n = 1 + log2 n (so C(n) in Theta(log n)).

(b) Master Theorem

a = 1, b = 2, so n^(log_b a) = n^(log2 1) = n^0 = 1. Since f(n) = 1 = Theta(n^0) = Theta(n^(log_b a)), **Case 2** applies and T(n) = Theta(n^(log_b a) log n) = **Theta(log n)**.

(c) Substitution (induction) for the exact worst case on any n

Claim: C(n) = floor(log2 n) + 1 for all n >= 1. *Base:* C(1) = 1 = floor(log2 1)+1. *Step:* assume it holds for all smaller values and let 2^k <= n < 2^(k+1), so floor(log2 n) = k. Then 2^(k-1) <= floor(n/2) < 2^k, hence floor(log2 floor(n/2)) = k-1, and

```
C(n) = C(floor(n/2)) + 1 = (k - 1 + 1) + 1 = k + 1 = floor(log2 n) + 1.   QED
```

(d) Verification against the experiment

Table 7.1 - Prediction vs observation

n	floor(log2 n) + 1 (predicted worst)	Max comparisons observed
1,000	10	10
10,000	14	14
100,000	17	17
1,000,000	20	20

Interpretation. The solution grows by one comparison every time n doubles. Going from 1,000 to 1,000,000 records (1,000 times more data) adds just 10 comparisons; the recursion depth never exceeds 20, so there is no risk of stack overflow, and the algorithm scales extremely well - but only for sorted data.

7.2 Merge sort (preprocessing): W(n) = 2W(n/2) + n - 1, W(1) = 0

Master Theorem: a = 2, b = 2, f(n) = n - 1 = Theta(n) = Theta(n^(log2 2)). Case 2 gives **Theta(n log n)**.

Iteration (n = 2^k):

```
W(n) = 2W(n/2) + (n-1)
   = 4W(n/4) + (n-2) + (n-1)
   = 2^i W(n/2^i) + i*n - (2^i - 1)
   i = log2 n :  W(n) = n*0 + n log2 n - (n - 1) = n log2 n - n + 1
```

Check: substituting W(n) = n log2 n - n + 1 on the right-hand side gives 2[(n/2)(log2 n - 1) - n/2 + 1] + n - 1 = n log2 n - n + 1. The experiment sorted 1,000,000 random numbers with 18,674,355 comparisons, between the best case (n/2)log2 n = 9.97 x 10^6 and the worst case n log2 n - n + 1 = 18.93 x 10^6. Random data is close to, but slightly below, the worst case: the known average for random input is about n log2 n - 1.25 n = 18.68 x 10^6, which matches the measurement.

Interpretation. The sort needs about n log2 n comparisons - for a million records, about 19 million - which is why preprocessing dominates for a small number of queries and is negligible when q is large.

Task 8 - Implementation and Testing

Environment and files

All algorithms are implemented in **Python 3.12.3** (x86-64 Linux container, single thread). Timing uses time.perf_counter. The source code lives in Task_8_Implementation_Testing/Source_Code:

Table 8.1 - Source files

File	Purpose
algorithms.py	linear_search, binary_search_recursive, merge_sort, merge_sorted, HashIndex (+ *_counted variants that return the number of comparisons)
data_gen.py	Reproducible dataset generator (seeded). 9-digit unique registration numbers, random storage order; present and absent query sets.
test_algorithms.py	12 unit tests (edge cases, equivalence of all algorithms, theoretical operation counts).
run_experiments.py	Benchmark for n = 1,000 / 10,000 / 100,000 / 1,000,000; writes results.json / results.csv.
design_challenge.py	Task 12 experiments: 50,000 searches/hour, daily update strategies, break-even.
make_graphs.py / make_screenshots.py	Produce the figures and the rendered output/source images used in the report.

Testing procedure (reproducible)

- **Same data for all algorithms.** For each n the same seeded list of n unique registration numbers (random order) is given to Algorithm A; the same list, sorted by merge sort, is given to B; the same list is indexed by C. Because the seed is fixed, every run generates the same data.
- **Same queries.** 2,000 successful keys (randomly chosen existing numbers) and 2,000 unsuccessful keys (guaranteed absent). Linear search is much slower, so it uses the first 300 queries (100 when n >= 100,000) for timing; its comparison counts use 1,000 queries.
- **Timing.** Mean time per search = (total time of the query set) / (number of queries); the median of 3 (linear) or 5 (binary, hash) repetitions is reported. Preprocessing is timed separately (median of 3 repeats; once for n = 1M). Counting uses separate *_counted functions so that counters do not disturb timings.
- **Correctness.** Every algorithm's result is checked against the others and against the ground truth in unit tests; the sorted output of merge sort is asserted equal to Python's sorted().

Table 8.2 - Test cases (all 12 unit tests pass)

Test case	Input	Expected result	Result
Empty database	[] , key 5	-1 for all three	Pass
Single element - present / absent	[7], key 7 / 8	0 / -1	Pass
Key is first / last record	8-element list	found at the correct position	Pass
Key below minimum / between / above maximum	keys 1, 500, 5000	-1 for all three	Pass
Every present key (small list)	8 keys	found, record equals key	Pass

Test case	Input	Expected result	Result
Merge sort correctness	n = 0,1,2,3,10,257,5000	equals sorted()	Pass
Linear counts	n = 1000	best 1, worst (last / absent) 1000	Pass
Binary worst-case bound	n = 1,2,7,8,1000,4096	max comparisons = floor(log2 n)+1	Pass
Counted == plain versions	n = 1000, 200 queries	same positions	Pass
Large random agreement	n = 20,000; 500 hits + 500 misses	all three agree with ground truth	Pass
Hash insert (daily update)	insert 42 into [1,2,3]	found; length 4	Pass
merge_sorted	two sorted lists	equals sorted(a+b)	Pass

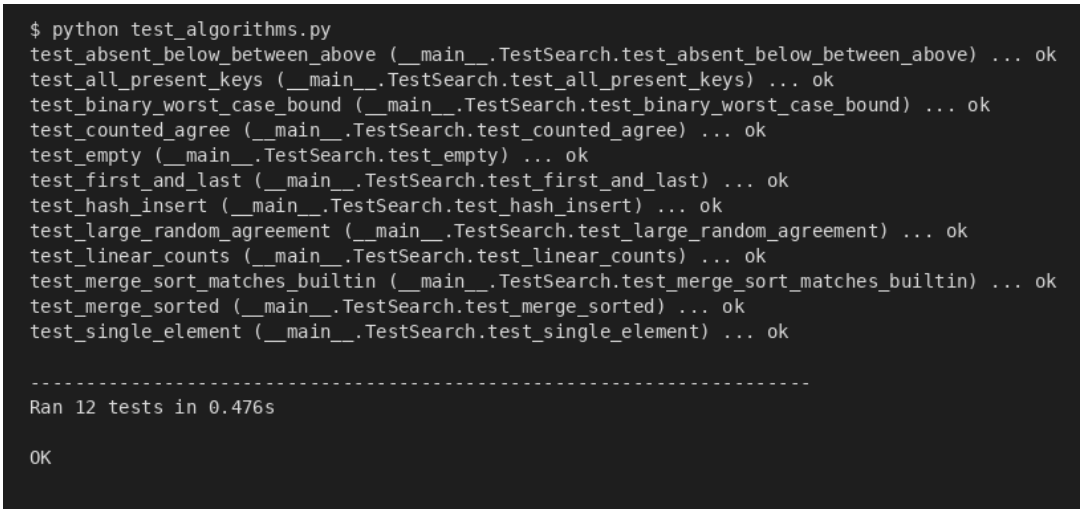


Figure 8.1 - Unit-test output (python test_algorithms.py)

The complete source code is included in the repository; rendered listings are in Report/Source_Code_Screenshots and in the appendix of this report.

How to run

```
cd Task_8_Implementation_Testing/Source_Code
python test_algorithms.py        # unit tests
python run_experiments.py        # Tasks 8-9 benchmark (about 2-3 minutes)
python design_challenge.py       # Task 12 experiments
python make_graphs.py            # figures (needs matplotlib)
```

Task 9 - Experimental Performance

All values below come from run_experiments.py (raw data: results.csv / results.json in Task_9_Experimental_Performance). Measured on one machine, so absolute values will differ on another computer; the **growth behaviour** is what matters.

Table 9.1 - Mean time per search (microseconds)

n	A Linear hit	B Binary hit	C Hash hit	A Linear miss	B Binary miss	C Hash miss
1,000	8.692	0.756	0.152	18.02	0.956	0.146
10,000	99.16	1.462	0.218	274.80	1.583	0.171
100,000	1,310.90	1.675	0.591	2,276.68	2.898	0.267
1,000,000	10,789.28	3.175	0.539	25,355.30	3.910	0.434

Table 9.2 - Key comparisons (basic operations)

n	A avg (hit)	A worst	B avg (hit)	B worst	C avg (hit)	C avg (miss)	C max chain
1,000	485	1,000	8.98	10	1.333	0.729	4
10,000	5,201	10,000	12.37	14	1.364	0.751	5
100,000	50,121	100,000	15.73	17	1.377	0.712	6
1,000,000	501,371	1,000,000	18.93	20	1.401	0.777	5

Table 9.3 - Preprocessing time and extra memory

n	Merge sort comparisons	Merge sort s	Built-in sorted() s	Hash build s	Sorted copy MB	Hash index MB
1,000	8,712	0.0008	0.0001	0.0002	0.01	0.09
10,000	120,390	0.0099	0.0010	0.0025	0.08	0.96
100,000	1,536,237	0.1783	0.0144	0.0398	0.80	9.65
1,000,000	18,674,355	2.5307	0.2741	1.1288	8.00	96.64

Table 9.4 - Break-even number of searches in measured time (successful-search timings)

n	Searches q so that sort pays for itself (B vs A)	Searches q so that hash build pays for itself (C vs A)
1,000	102	25
10,000	101	25
100,000	136	30
1,000,000	235	105

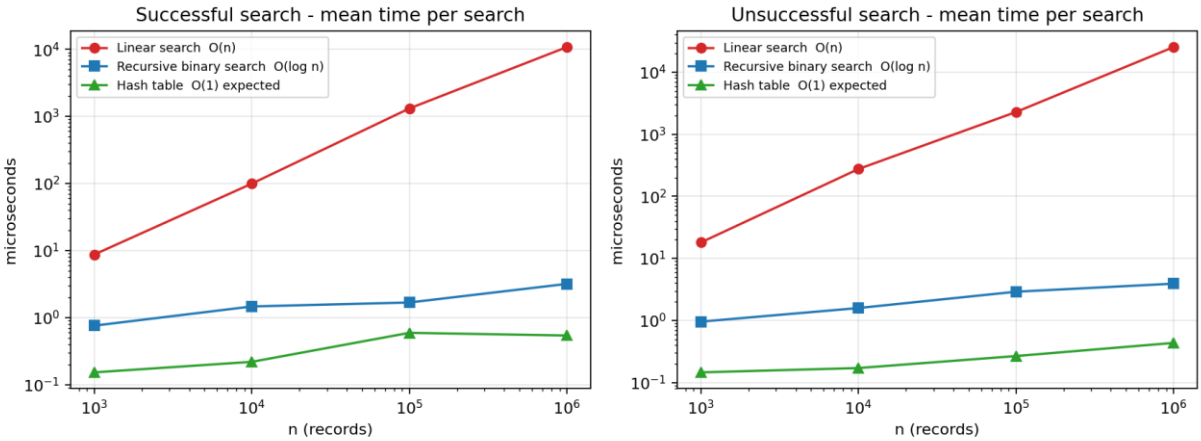


Figure 9.1 - Search time vs n (log-log). Linear slope ~1; binary and hash almost flat.

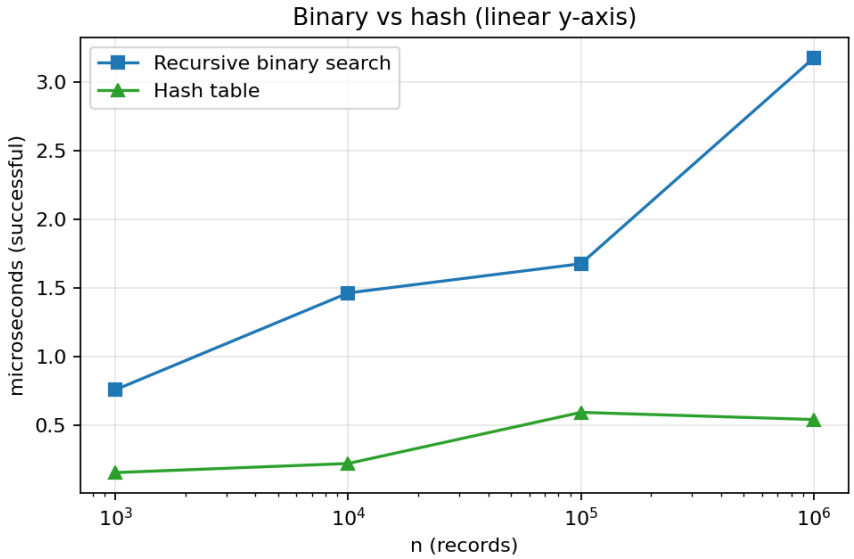


Figure 9.2 - Binary search vs hash table on a linear y-axis

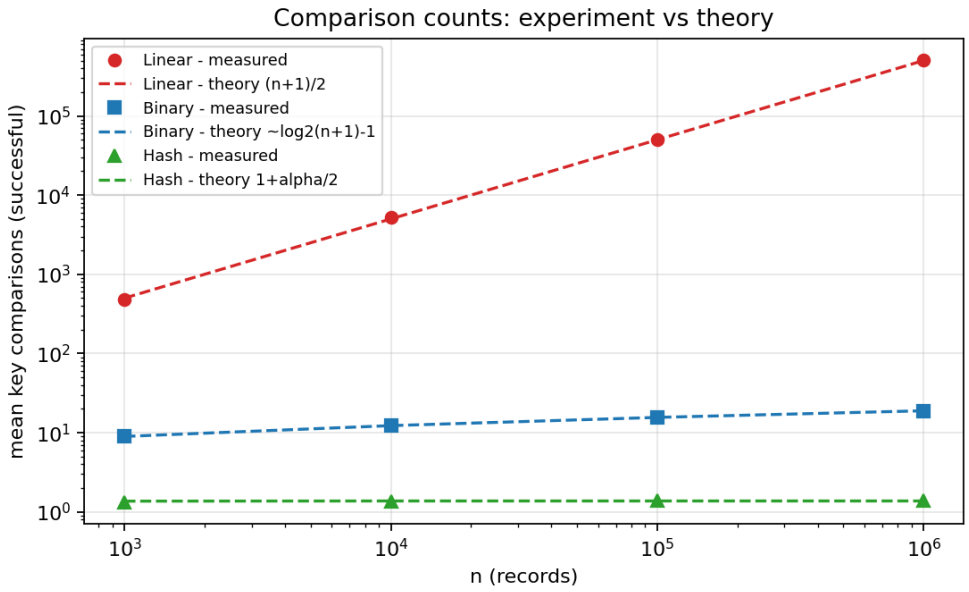


Figure 9.3 - Measured comparison counts against the theoretical formulas

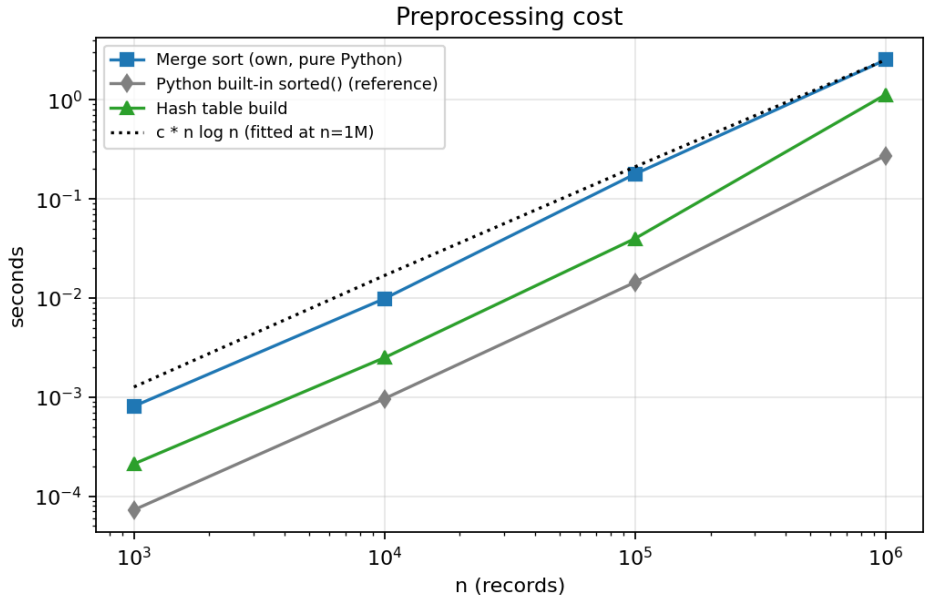


Figure 9.4 - Preprocessing time vs n

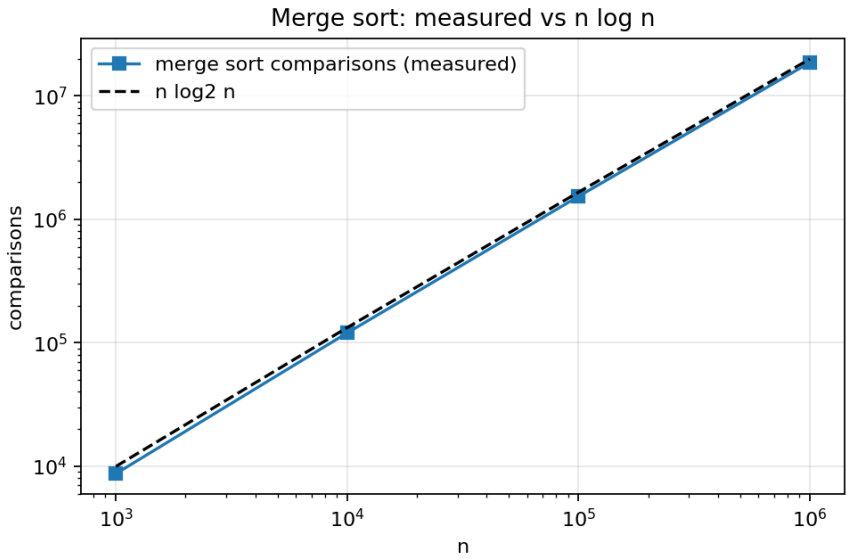


Figure 9.5 - Merge sort comparisons vs n log2 n

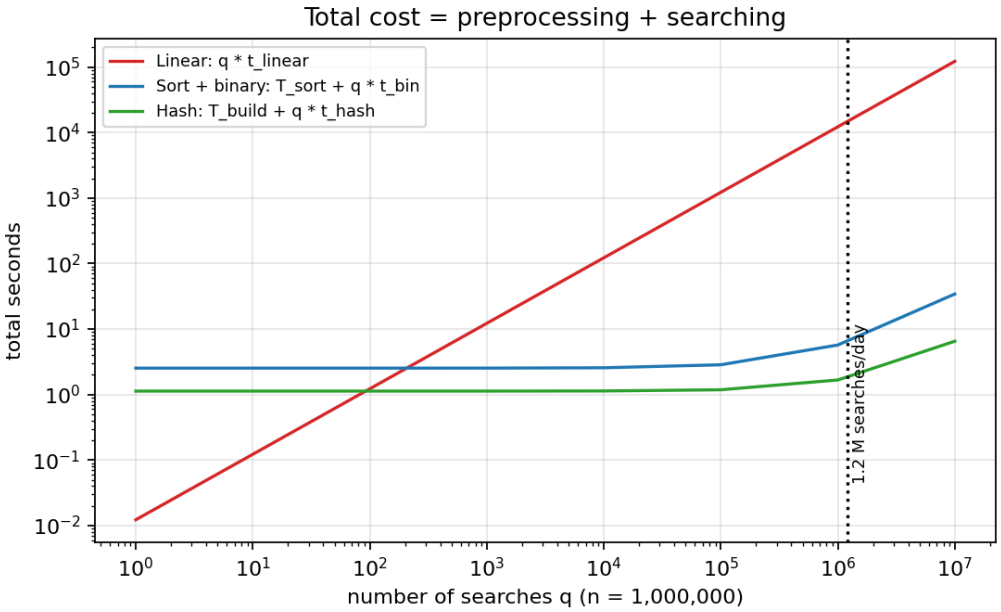


Figure 9.6 - Total cost (preprocessing + q searches) at n = 1,000,000

```
$ python run_experiments.py

=== n = 1,000 ===
  done: linear 8.7 us | binary 0.76 us | hash 0.152 us (successful search, mean)

=== n = 10,000 ===
  done: linear 99.2 us | binary 1.46 us | hash 0.218 us (successful search, mean)

=== n = 100,000 ===
  done: linear 1310.9 us | binary 1.67 us | hash 0.591 us (successful search, mean)

=== n = 1,000,000 ===
  done: linear 10789.3 us | binary 3.17 us | hash 0.539 us (successful search, mean)

-----------------------------------------------------------------------------------------------
MEAN TIME PER SUCCESSFUL SEARCH (microseconds)  /  MEAN COMPARISONS (successful)
-----------------------------------------------------------------------------------------------
         n |    Linear us  Binary us   Hash us |  Linear cmp  Binary cmp  Hash cmp
     1,000 |         8.69      0.756     0.152 |       485.2        8.98     1.333
    10,000 |        99.16      1.462     0.218 |      5201.0       12.37     1.364
   100,000 |      1310.90      1.675     0.591 |     50121.5       15.73     1.377
 1,000,000 |     10789.28      3.175     0.539 |    501371.1       18.93     1.401
-----------------------------------------------------------------------------------------------
PREPROCESSING (seconds)  /  WORST-CASE COMPARISONS OBSERVED
-----------------------------------------------------------------------------------------------
         n |  MergeSort s  HashBuild s | Linear worst Binary worst Hash worst
     1,000 |       0.0008       0.0002 |        1,000          10          4
    10,000 |       0.0099       0.0025 |       10,000          14          5
   100,000 |       0.1783       0.0398 |      100,000          17          6
 1,000,000 |       2.5307       1.1288 |    1,000,000          20          5
-----------------------------------------------------------------------------------------------

Saved results.json / results.csv
```

Figure 9.7 - Console output of run_experiments.py

Task 10 - Theory vs Experiment

Operation counts agree almost exactly with theory

Table 10.1 - Theoretical vs observed counts

Quantity (n = 1,000,000)	Theory	Measured	Agreement
Linear, successful average comparisons	(n+1)/2 = 500,000.5	501,371	0.27% difference (sampling error of 1,000 random keys)
Linear, worst case	n = 1,000,000	1,000,000	exact
Binary, worst case	floor(log2 n)+1 = 20	20	exact
Binary, successful average	18.93	18.93	within sampling error
Hash, successful average	1 + alpha/2 = 1.375 (alpha = 0.750)	1.401	close; difference is sampling error of 2,000 keys
Hash, unsuccessful average	alpha = 0.750	0.777	close
Merge sort comparisons	n log2 n - n + 1 = 18.93 x 10^6 (worst); random-data average ~18.67 x 10^6	18,674,355	between best and worst; matches random-data average

Running time: the growth rates agree, constants and memory effects explain the deviations

Table 10.2 - Growth over a 1,000-fold increase in n

Algorithm	Theoretical growth of time, n: 1K to 1M	Observed growth of time (successful)	Per-operation cost at 1K vs 1M
A Linear	1,000 x	1,241 x	17.9 ns vs 21.5 ns per comparison
B Binary	log2(1M)/log2(1K) = 2.0 x	4.2 x	84 ns vs 168 ns per level
C Hash	1 x	3.5 x	1.3 vs 1.4 comparisons

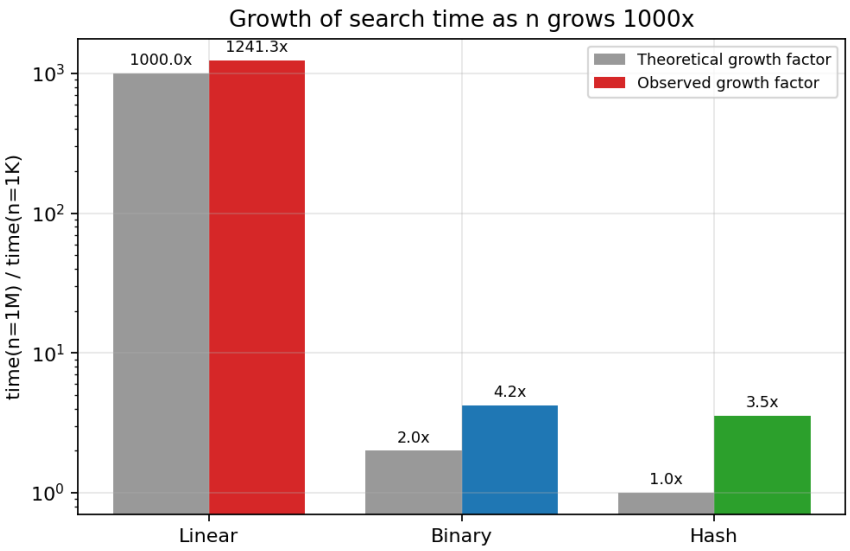


Figure 10.1 - Theoretical vs observed growth factor of search time

- **Linear search** follows the prediction best: its time rises by about 10x for every 10x increase in n (slope 1 on the log-log plot) and the observed 1000-fold growth is about 1.2x larger than predicted. The remaining excess is a memory effect: at n = 1,000 the whole list fits in cache, while at n = 1,000,000 the pointers and the integer objects they refer to occupy tens of megabytes. The loop is also faster per element at the smallest sizes because fixed start-up cost is amortised over fewer elements.
- **Binary search** grows much more slowly than linear search and is logarithmic in the *number of comparisons* (8.98, 12.37, 15.73, 18.93), which agrees with theory, but its *time* grows 4.2x rather than 2x. The cost per level rose from about 84 ns to 168 ns: with a sorted array of 1M elements the first probes hit different memory pages and miss the CPU cache, whereas a 1,000-element array is entirely cache resident. Recursion overhead in Python is a constant per level and cannot cause growth. So the asymptotic model (every comparison costs the same) is violated by the memory hierarchy, not by the algorithm.
- **Hash table.** The comparison count is constant (1.33 to 1.40), exactly as predicted, yet the time grew 3.5x. Here too, the lookup touches three or four distinct memory locations (bucket slot, chain list, record, key object) which are cache-resident for small n but not for n = 10^6. The differences between consecutive sizes (e.g. 0.59 us at n = 100,000 vs 0.54 us at n = 1,000,000) are within run-to-run variation of a few tenths of a microsecond; for such short operations the timer resolution and garbage-collector/OS noise are of the same order as the effect.
- **Preprocessing.** Merge sort time grew by 12x, 18x and 14x for successive 10x increases of n (n log n predicts about 12x); the hash build grew by 12x, 16x and 28x (theory: 10x). The extra growth in the hash build is memory allocation: creating about 700,000 Python list objects for the chains at n = 10^6 is slower per record than the few hundred lists at n = 10^3, and the larger working set causes cache misses.

Which algorithm scales best?

Scalability ordering in both theory and experiment is **C (hash) > B (binary) > A (linear)**. At n = 1,000,000 a successful hash search took about 0.5 us, binary search about 3 us and linear search about 10,800 us (10.8 ms) -

the linear search was about 3,400 times slower than binary search and about 20,000 times slower than the hash lookup.

Reasons for remaining differences between theory and experiment

- **Constant factors and the interpreter.** Big-O ignores constants; in Python one loop iteration costs about 20 ns, one recursive call about 80-170 ns, one hash lookup about 0.3-0.6 us. A compiled language would change the constants but not the growth rates.
- **Memory hierarchy (cache / TLB misses)** as explained above - the RAM-model assumption of uniform access cost does not hold.
- **Sampling and measurement noise.** Averages over 100-2,000 random keys have statistical error (1,000 sampled linear searches carry a 0.3% error); sub-microsecond timings include loop and timer overhead; the experiment ran on a shared machine.
- **Assumption of uniform hashing.** The theoretical alpha-based figures assume ideal hashing; the measured values are slightly higher, as is expected for a real hash function.
- **Operation weighting.** The unsuccessful linear search took between 1.7x and 2.8x as long as the successful one (2.35x at n = 1M) against the 2x predicted from n vs (n+1)/2; the spread comes from the small number of timed queries and from the sampling of where the successful keys happen to lie.

Task 11 - Final Design Recommendation

Recommendation for the original (general-purpose) problem: keep the registration numbers in a sorted array and use binary search; sort once with merge sort (or the system sort) and re-merge new records in batches. The hash table is the strongest alternative and is adopted in Task 12 once the workload is known.

Why this choice - evidence from analysis and experiment

Table 11.1 - Criteria-based comparison

Criterion	A Linear	B Sort + binary	C Hash
Search cost, theory	Theta(n)	Theta(log n), worst case guaranteed	Theta(1) expected, Theta(n) worst
Measured search at n = 1M (hit / miss)	10,789 / 25,355 us	3.17 / 3.91 us	0.54 / 0.43 us
Preprocessing (theory)	none	Theta(n log n)	Theta(n)
Preprocessing measured, n = 1M	0	2.53 s (own merge sort); 0.27 s with built-in sort	1.13 s
Preprocessing paid back after (searches)	-	~235 searches	~105 searches
Extra memory at n = 1M	0	8 MB (one pointer per record)	97 MB (about 12x more)
Supports ordered / range / prefix queries (e.g. all students of one batch)	no (full scan)	yes - contiguous range	no
Depends on hash quality / worst-case guarantee	n/a	no - deterministic	yes
Practical as n grows	No - cost grows with n	Yes - 20 -> 24 comparisons for 1M -> 10M	Yes - if table is resized with n

- **Why not linear search?** At 1,000,000 records one lookup takes 10.8 ms on average and 25.4 ms when the number is not present. Its only advantage - no preprocessing - disappears after roughly 100-200 searches (Table 9.4), a tiny number for a university system.
- **Why binary search rather than the (faster) hash table in the *general* case?** (i) Guaranteed worst case of 20 comparisons for 10^6 records, whatever the key pattern; (ii) 12 times less extra memory (8 MB vs 97 MB); (iii) a sorted array answers range/prefix questions that administrators actually ask (all registration numbers of one admission year); (iv) it is about 6x slower than the hash table in the single-key benchmark (2.5x in the mixed 50,000-request run of Task 12) - a difference of only a few microseconds per search; (v) it only needs a comparison order on keys. When nothing is known about update or query frequency, this is the most robust choice.

Trade-offs and limitations

- **Preprocessing cost.** Sorting 1M records took 2.5 s with the simple merge sort (the pure-Python built-in sort needs 0.27 s). It must be repeated or repaired when records are added: inserting into the middle of a sorted array costs O(n) per record, so new records should be collected and merged in batches (sort the k new keys, then one linear merge - 0.24 s for 2,000 keys at n = 1M, Task 12).
- **Memory/caching.** Binary search touches scattered memory locations; at n = 10^6 each level cost about twice as much as at n = 10^3 (Task 10).
- **Scalability.** Extrapolating the measured n log n behaviour, sorting 10 million records would take about 30 s in this implementation (extrapolation, not measured); a search would need 24 comparisons. Beyond main-memory sizes the same idea generalises to disk-based B-trees, which are the multi-way relatives of binary search.
- **Not recommended** if the data change continuously (every insert needs a re-sort or a tree structure), or if memory is not a concern and only exact-match lookups are needed - the situation analysed in Task 12.

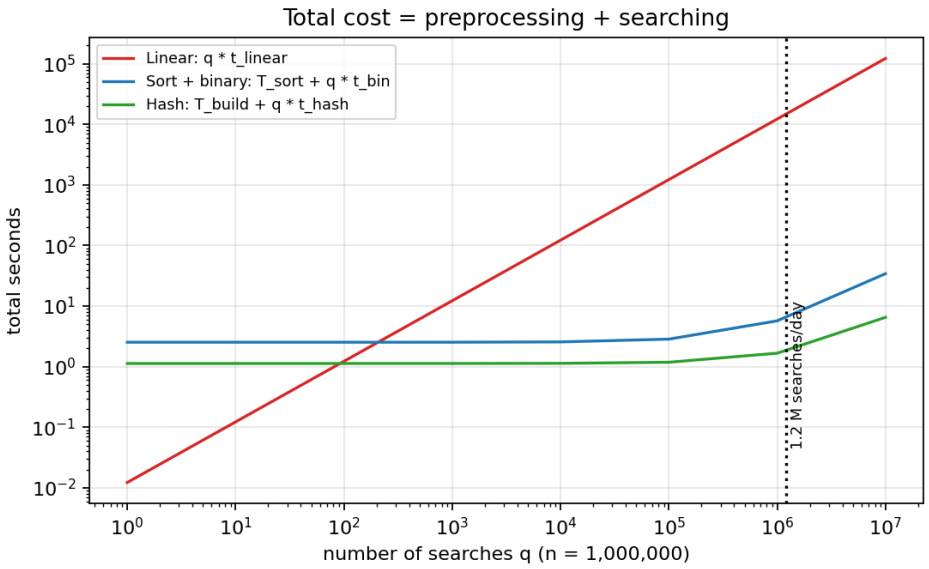


Figure 11.1 - Total cost of each design as the number of searches grows (n = 1M)

DESIGN CHALLENGE

Task 12 - Design Challenge

Changed requirement: 50,000 search requests per hour = 13.9 requests per second = 1,200,000 per day on a database of 1,000,000 records, with new records added **once per day**. **Decision: MODIFY** - the sorted-array design is kept as the ordered master copy, but exact-match lookups are redesigned to use a hash index that is rebuilt/extended in a nightly batch.

A. Impact analysis (2 marks)

- **What changed.** The original problem gave no information about how many searches there would be; q was an unknown symbol. Now q is known: about 1,200,000 searches per day, i.e. about **1.2 searches per record per day**, and the data are static for 24 hours at a time.
- **Assumptions affected.** (1) 'Preprocessing cost must be weighed against a handful of searches' - no longer true: the nightly preprocessing (2.6 s sort or 1.5 s hash build) is shared by 1.2 million searches, i.e. 2.2 us or 1.2 us per search. (2) 'The database may change at any time and the structure must handle frequent updates' - no longer true; updates are a daily batch.
- **Why search frequency matters now.** Total cost = preprocessing + q x (cost per search). With q = 1.2 million/day the per-search term dominates everything. At the linear-search speed of 14.9 ms the daily search work would be about 5.0 CPU-hours, versus 5.1 s (binary) or 2.3 s (hash). A change in the constant or order of the per-search cost is multiplied by 1.2 million.
- **Why infrequent updates matter.** Between updates the data never change, so an index can be built once and used read-only: no locking, no resizing in the hot path, the expensive build is amortised over a whole day, and a perfect-fit table (load factor 0.75) can be sized for tomorrow's n. Structures that are fast to search but slow to update (sorted array, static hash) are exactly the right trade-off.

B. Evaluation of the original design

One simulated hour (50,000 searches, 90% successful) on 1,000,000 records was run (design_challenge.py):

Table 12.1 - One hour of traffic (n = 1,000,000)

Design	Mean latency	p50	p99	Max seen	CPU for 50,000 searches	Share of one core	Capacity per core (req/s)
A Linear (measured on 300, extrapolated)	14.9 ms	15.6 ms	32.9 ms	33.5 ms	747 s	20.8%	67
B Binary search (measured, 50,000)	4.08 us	3.80 us	9.48 us	438 us	0.21 s	0.0059%	245,062
C Hash index (measured, 50,000)	1.60 us	1.31 us	3.38 us	1417 us	0.10 s	0.0027%	624,906

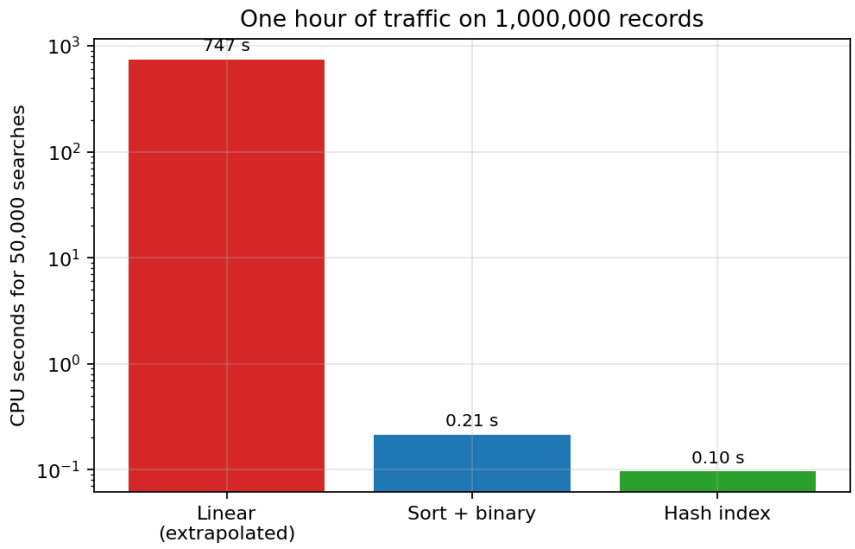


Figure 12.1 - CPU time for one hour of traffic (log scale)

- **Original recommendation (sorted array + binary search) under 50,000 searches/hour:** entirely adequate - 0.21 s of CPU per hour (about 0.006% of one core), p99 latency 9.5 us. The load is nowhere near a bottleneck.
- **Preprocessing acceptable?** Yes: 2.6 s per day for the full sort, or 0.24 s to merge 2,000 new records into the existing sorted array (sort the k new keys, then one O(n + k) merge) - negligible next to a 24-hour day (Table 12.2).
- **Could another approach be better?** Linear search is technically feasible on one core (21% utilisation, 15 ms mean and up to 33-42 ms latency), but it burns 5.0 CPU-hours a day, has only 4.8x head-room over the average load (a 10x peak, 139 req/s, would need more than two cores), and degrades linearly as the university grows - rejected. The hash index is the better of the two fast designs: 2.5x lower mean latency, 2.8x lower p99, a build that is 1.8x cheaper than the sort, and O(1) insertion.

C. Adaptation / redesign

Decision: MODIFY. The workload is read-dominated and the updates are batched and rare, which favours a precomputed, read-only index with the fastest possible lookup.

- **Revised strategy.** (1) Keep the sorted array of registration numbers as the *ordered master copy* (range/prefix queries, audit, fallback). (2) Build a hash index (separate chaining, load factor 0.75, multiplicative hash) over the master copy. (3) All exact-match searches (the 50,000/hour) go to the hash index. (4) Once per day a batch job (a) sorts the k new records (O(k log k)) and merges them with the master array (O(n + k)); (b) inserts the k new keys into the hash index (O(k) expected) or rebuilds it in O(n); (c) swaps the new snapshot in atomically, so concurrent readers never need locks.
- **What changed relative to the original design.** The search path changed from O(log n) comparison-based to O(1) expected; the per-day update path changed from 're-sort everything' (O(n log n)) to 'merge a small batch' (O(n + k)) plus incremental hash insertion (O(k)); memory increases from about 8 MB to about 105 MB (an acceptable price for a server).

Table 12.2a - Effect on complexity

Quantity	Original (sorted array + binary)	Modified (hash index + sorted master)
Search time (worst / expected)	Theta(log n) / Theta(log n)	Theta(n) worst, **Theta(1) expected**
Daily update, k new records	re-sort O(n log n) or merge O(n + k)	hash insert **O(k)**, master merge O(n + k)
Searches per day q = 1.2 x 10^6: total search comparisons	q log2 n = 2.4e+07	q (1+alpha/2) = 1.7e+06
Space	Theta(n)	Theta(n) (about 13x larger constant)
Range / prefix queries	yes	yes (through the master copy)

Table 12.2 - Daily update strategies

Strategy for adding 2,000 records to 1,000,000	Time (s)
Append + full re-sort (own merge sort)	2.764
Append + full re-sort (built-in sort)	0.280
Sort 2,000 new keys, then merge with sorted master O(n + k)	0.236
Hash index: incremental insert of the 2,000 keys O(k)	0.007
Hash index: full rebuild O(n)	1.585

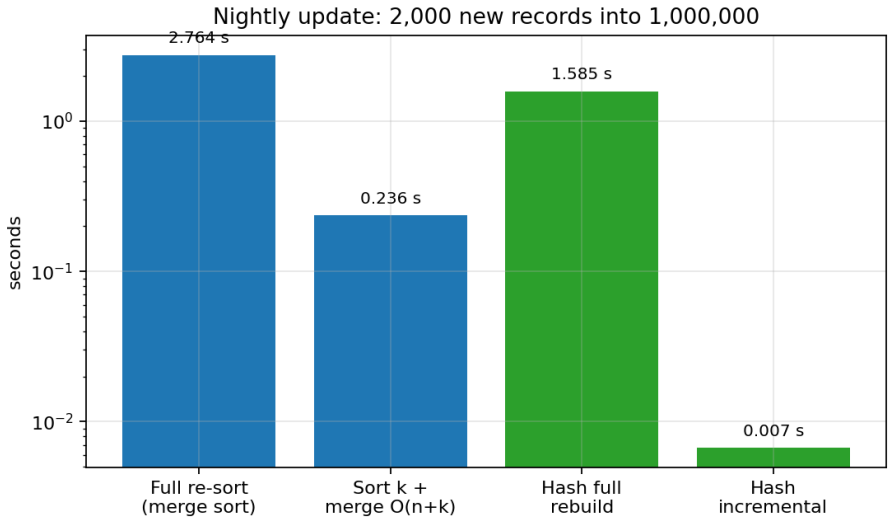


Figure 12.2 - Nightly update cost for 2,000 new records (log scale)

```
$ python design_challenge.py   (summary)

One hour of traffic = 50,000 searches on n = 1,000,000   (13.9 req/s)
  linear  mean   14949.76 us  p50   15563.10  p99   32918.65  max    33474.3  hour   747.49 s  (20.764% of 1 core)
  binary  mean       4.08 us  p50       3.80  p99       9.48  max      437.9  hour     0.21 s  (0.006% of 1 core)
  hash    mean       1.60 us  p50       1.31  p99       3.38  max     1417.1  hour     0.10 s  (0.003% of 1 core)

Preprocessing: merge sort 2.62 s | hash build 1.48 s
Daily update with 2,000 new records:
  resort_full_mergesort_s           2.764 s
  resort_full_timsort_s             0.280 s
  sort_k_then_merge_s               0.236 s
  hash_incremental_insert_s         0.007 s
  hash_full_rebuild_s               1.585 s
Searches needed to amortise sort: 214 | hash build: 121
```

Figure 12.3 - Console output of design_challenge.py

D. Evidence-based defence

- **Search frequency.** 13.9 requests/s means a budget of 72 ms per request per core. The hash index uses 1.6 us (about 45,000 times below the budget); binary search 4.1 us; linear search 14.9 ms (about one fifth of the budget). One core running hash lookups could theoretically serve about 624,906 requests per second; binary search 245,062; linear search only 67.
- **Preprocessing cost.** The hash build (1.48 s) is paid back after about 121 searches - about 0.01% of one day's traffic - versus about 214 for the sort (2.62 s). Spread over 1.2 million daily searches the build costs about 1.2 us per search.
- **Dataset size and growth.** Search comparisons are 1.4 (hash) vs 19-20 (binary) vs 500,000 (linear) at n = 10^6, and the hash and binary figures barely change if n grows tenfold, whereas linear search would need 10x more time (about 150 ms mean at 10^7 records, extrapolated).
- **Mathematics.** Total daily comparison count: linear n/2 x q = 6 x 10^11; binary n log n + q log n = 2 x 10^7 + 2.4 x 10^7 = 4.4 x 10^7; hash n + 1.4 q = 10^6 + 1.7 x 10^6 = 2.7 x 10^6 (details in Task 3). The hash design performs the fewest basic operations.
- **Trade-offs honestly stated.** Both fast designs use well under 0.01% of a core at the stated load, so the *absolute* gain from hashing over binary search is small (about 2.5 us per search); it is chosen for lower tail latency, cheaper nightly update, and head-room for peaks and growth. Costs: about 13x more memory (97 MB), expected-case (not worst-case) guarantee, no ordering - which is why the sorted master copy is kept. If memory were severely limited, or no other requirement than 50,000 searches/hour existed and simplicity were preferred, **retaining** binary search would be an equally defensible decision; linear search is not.
- **Limitations of the experiment.** Timings are from one machine (single thread, Python), a 90/10 hit/miss mix, uniformly random keys and no network, disk or concurrency effects; linear search was extrapolated from 300 measured queries. The conclusions rest on the order-of-magnitude differences, which are far larger than any of these effects.

Conclusion

- **Theory.** Linear search is Theta(n); recursive binary search satisfies C(n) = C(floor(n/2)) + 1 and is Theta(log n) after a Theta(n log n) sort; a hash table gives Theta(1) expected lookup after a Theta(n) build.
- **Experiment.** Operation counts match the formulas almost exactly (e.g. binary worst case 20 = floor(log2 10^6) + 1; linear average 501,371 vs 500,000.5). Running times follow the predicted ordering and growth; the small deviations (binary and hash times growing faster than 2x and 1x) are caused by CPU-cache effects, not by the algorithms.
- **Design.** For the original, general problem the sorted array with binary search is the most robust choice; once it is known that searches are frequent (1.2 million per day) and updates are daily batches, a hash index over the sorted master copy gives the lowest latency and the cheapest daily maintenance. Linear search is not suitable for a million-record system.
- **Lesson.** The lowest Big-O is not automatically the best design; preprocessing, memory, worst-case guarantees, ordered queries, update pattern and the number of searches all decide, and the experiment showed where the asymptotic model and the real machine differ.

Appendix A - Repository structure

```
CO1_Assignment/
|-- README.md
|-- Task_1_Algorithm_Design/          (design, pseudocode)
|-- Task_2_Basic_Operation/
|-- Task_3_Mathematical_Complexity/
|-- Task_4_Case_Analysis/
|-- Task_5_Asymptotic_Analysis/
|-- Task_6_Recurrence_Formulation/
|-- Task_7_Recurrence_Solution/
|-- Task_8_Implementation_Testing/
|   |-- README.md
|   `-- Source_Code/  algorithms.py  data_gen.py  test_algorithms.py
|                     run_experiments.py  design_challenge.py  make_graphs.py  make_screenshots.py
|-- Task_9_Experimental_Performance/    results.csv  results.json
|-- Task_10_Theory_vs_Experiment/
|-- Task_11_Final_Recommendation/
|-- Task_12_Design_Challenge/          design_challenge_results.json
`-- Report/
    |-- Source_Code_Screenshots/
    |-- Output_Screenshots/
    |-- Analysis/
    |-- Tables_and_Graphs/
    `-- Documentation/                 CO1_Assignment_Report.docx
```

Appendix B - Source code listings (rendered)

```
"""
algorithms.py - Three distinct strategies for locating a registration number.

  1. linear_search            : ITERATIVE  - brute force sequential scan (no preprocessing)
  2. binary_search_recursive  : RECURSIVE  - decrease-and-conquer on a sorted array
                                (preprocessing: merge_sort, divide-and-conquer)
  3. HashIndex                : ITERATIVE  - hashing with separate chaining (space-time trade-off)
                                (preprocessing: build the table in O(n))

Every search has two forms:
  * a plain version used for TIMING (no instrumentation overhead)
  * a *_counted version that returns (position, comparisons) used for operation counts
"""
import sys

sys.setrecursionlimit(10_000)


# ---------------------------------------------------------------------------
# 1. Linear search (iterative, brute force)
# ---------------------------------------------------------------------------
def linear_search(a, key):
    """Return index of key in unsorted list a, or -1."""
    for i, v in enumerate(a):
        if v == key:
            return i
    return -1


def linear_search_counted(a, key):
    comps = 0
    for i, v in enumerate(a):
        comps += 1                      # basic operation: a[i] == key
        if v == key:
            return i, comps
    return -1, comps


# ---------------------------------------------------------------------------
# 2. Binary search (recursive, decrease-and-conquer) + merge sort preprocessing
# ---------------------------------------------------------------------------
def binary_search_recursive(a, key, lo=0, hi=None):
    """Return index of key in SORTED list a, or -1."""
    if hi is None:
        hi = len(a) - 1
    if lo > hi:                          # base case: empty sub-array
        return -1
    mid = (lo + hi) // 2
    v = a[mid]
    if v == key:
        return mid
    if key < v:
        return binary_search_recursive(a, key, lo, mid - 1)
    return binary_search_recursive(a, key, mid + 1, hi)

```

src algorithms 1

```
 57  def binary_search_counted(a, key, lo=0, hi=None, _c=0):
 58      """Counts one three-way comparison (key vs a[mid]) per recursive call."""
 59      if hi is None:
 60          hi = len(a) - 1
 61      if lo > hi:
 62          return -1, _c
 63      mid = (lo + hi) // 2
 64      _c += 1                             # basic operation: compare key with a[mid]
 65      v = a[mid]
 66      if v == key:
 67          return mid, _c
 68      if key < v:
 69          return binary_search_counted(a, key, lo, mid - 1, _c)
 70      return binary_search_counted(a, key, mid + 1, hi, _c)
 71
 72
 73  def merge_sort(a):
 74      """Top-down merge sort. Returns (sorted_list, comparisons). T(n)=2T(n/2)+Theta(n)."""
 75      n = len(a)
 76      if n <= 1:
 77          return list(a), 0
 78      mid = n // 2
 79      left, cl = merge_sort(a[:mid])
 80      right, cr = merge_sort(a[mid:])
 81      out = []
 82      i = j = 0
 83      nl, nr = len(left), len(right)
 84      c = 0
 85      while i < nl and j < nr:
 86          c += 1                          # basic operation of sorting
 87          if left[i] <= right[j]:
 88              out.append(left[i]); i += 1
 89          else:
 90              out.append(right[j]); j += 1
 91      if i < nl:
 92          out.extend(left[i:])
 93      if j < nr:
 94          out.extend(right[j:])
 95      return out, cl + cr + c
 96
 97
 98  def merge_sorted(a, b):
 99      """Linear-time merge of two sorted lists (used for the daily batch update)."""
100      out = []
101      i = j = 0
102      na, nb = len(a), len(b)
103      while i < na and j < nb:
104          if a[i] <= b[j]:
105              out.append(a[i]); i += 1
106          else:
107              out.append(b[j]); j += 1
108      out.extend(a[i:]); out.extend(b[j:])
109      return out
110
```

src algorithms 2

```
# ---------------------------------------------------------------------------
# 3. Hash index with separate chaining (iterative)
# ---------------------------------------------------------------------------
def next_prime(x):
    def is_prime(p):
        if p < 2:
            return False
        if p % 2 == 0:
            return p == 2
        d = 3
        while d * d <= p:
            if p % d == 0:
                return False
            d += 2
        return True
    while not is_prime(x):
        x += 1
    return x


_KNUTH = 2654435761            # Knuth multiplicative constant (~2^32 / golden ratio)


class HashIndex:
    """Hash table (separate chaining). Chains store RECORD INDICES into the records list,
    exactly like a database index that points to a row; a lookup therefore compares
    records[idx] with the key. Load factor alpha = n / m is kept near 0.75."""

    def __init__(self, records, load=0.75):
        self.records = records
        self.m = next_prime(int(len(records) / load) + 1)
        self.table = [None] * self.m
        self.build()

    def _h(self, key):
        return ((key * _KNUTH) & 0xFFFFFFFF) % self.m

    def build(self):
        t, m, rec = self.table, self.m, self.records
        for i, k in enumerate(rec):
            h = ((k * _KNUTH) & 0xFFFFFFFF) % m
            b = t[h]
            if b is None:
                t[h] = [i]
            else:
                b.append(i)

    def insert(self, key):
        """Append a new record and index it: O(1) expected."""
        idx = len(self.records)
        self.records.append(key)
        h = self._h(key)
        if self.table[h] is None:
            self.table[h] = [idx]
```

src algorithms 3

```
        else:
            self.table[h].append(idx)

    def search(self, key):
        b = self.table[((key * _KNUTH) & 0xFFFFFFFF) % self.m]
        if b is None:
            return -1
        rec = self.records
        for idx in b:
            if rec[idx] == key:
                return idx
        return -1

    def search_counted(self, key):
        """Returns (index, key comparisons inside the chain)."""
        b = self.table[self._h(key)]
        if b is None:
            return -1, 0
        c = 0
        for idx in b:
            c += 1                       # basic operation: records[idx] == key
            if self.records[idx] == key:
                return idx, c
        return -1, c

    def chain_stats(self):
        lens = [len(b) for b in self.table if b is not None]
        return {"m": self.m, "n": len(self.records), "alpha": len(self.records) / self.m,
                "used_buckets": len(lens), "max_chain": max(lens) if lens else 0}

```

src algorithms 4

```python
"""Unit tests (python test_algorithms.py  or  python -m unittest -v)."""
import random
import unittest
from algorithms import (linear_search, binary_search_recursive, merge_sort, merge_sorted,
                        HashIndex, linear_search_counted, binary_search_counted)
from data_gen import make_dataset, make_queries


class TestSearch(unittest.TestCase):
    def setUp(self):
        self.data = [905, 120, 777, 431, 654, 999, 101, 310]
        self.sorted, _ = merge_sort(self.data)
        self.h = HashIndex(list(self.data))

    # --- boundary / special cases -------------------------------------
    def test_empty(self):
        self.assertEqual(linear_search([], 5), -1)
        self.assertEqual(binary_search_recursive([], 5), -1)
        self.assertEqual(HashIndex([]).search(5), -1)

    def test_single_element(self):
        self.assertEqual(linear_search([7], 7), 0)
        self.assertEqual(binary_search_recursive([7], 7), 0)
        self.assertEqual(HashIndex([7]).search(7), 0)
        self.assertEqual(binary_search_recursive([7], 8), -1)

    def test_first_and_last(self):
        self.assertEqual(linear_search(self.data, 905), 0)
        self.assertEqual(linear_search(self.data, 310), 7)
        self.assertEqual(self.sorted[binary_search_recursive(self.sorted, 101)], 101)
        self.assertEqual(self.sorted[binary_search_recursive(self.sorted, 999)], 999)

    def test_absent_below_between_above(self):
        for k in (1, 500, 5000):
            self.assertEqual(linear_search(self.data, k), -1)
            self.assertEqual(binary_search_recursive(self.sorted, k), -1)
            self.assertEqual(self.h.search(k), -1)

    def test_all_present_keys(self):
        for k in self.data:
            self.assertEqual(self.data[linear_search(self.data, k)], k)
            self.assertEqual(self.sorted[binary_search_recursive(self.sorted, k)], k)
            self.assertEqual(self.h.records[self.h.search(k)], k)

    # --- sorting ------------------------------------------------------
    def test_merge_sort_matches_builtin(self):
        for n in (0, 1, 2, 3, 10, 257, 5000):
            d = make_dataset(n) if n else []
            s, _ = merge_sort(d)
            self.assertEqual(s, sorted(d))

    def test_merge_sorted(self):
        a, b = sorted(random.sample(range(10000), 50)), sorted(random.sample(range(10000), 20))
        self.assertEqual(merge_sorted(a, b), sorted(a + b))

```

src tests 1

```
    # --- counted versions agree with plain versions ------------------------
    def test_counted_agree(self):
        d = make_dataset(1000)
        s, _ = merge_sort(d)
        h = HashIndex(list(d))
        hits, misses = make_queries(d, 100)
        for k in hits + misses:
            self.assertEqual(linear_search_counted(d, k)[0], linear_search(d, k))
            self.assertEqual(binary_search_counted(s, k)[0], binary_search_recursive(s, k))
            self.assertEqual(h.search_counted(k)[0], h.search(k))

    # --- worst-case operation counts match theory --------------------------
    def test_linear_counts(self):
        d = make_dataset(1000)
        self.assertEqual(linear_search_counted(d, d[0])[1], 1)           # best
        self.assertEqual(linear_search_counted(d, d[-1])[1], 1000)       # worst (last)
        self.assertEqual(linear_search_counted(d, -1)[1], 1000)          # worst (absent)

    def test_binary_worst_case_bound(self):
        import math
        for n in (1, 2, 7, 8, 1000, 4096):
            s = list(range(0, 2 * n, 2))
            bound = math.floor(math.log2(n)) + 1
            worst = max(binary_search_counted(s, k)[1] for k in range(-1, 2 * n + 1))
            self.assertLessEqual(worst, bound)
            self.assertEqual(worst, bound)

    # --- cross-check on a large random dataset -----------------------------
    def test_large_random_agreement(self):
        d = make_dataset(20000)
        s, _ = merge_sort(d)
        h = HashIndex(list(d))
        hits, misses = make_queries(d, 500)
        for k in hits:
            self.assertGreaterEqual(linear_search(d, k), 0)
            self.assertGreaterEqual(binary_search_recursive(s, k), 0)
            self.assertGreaterEqual(h.search(k), 0)
        for k in misses:
            self.assertEqual(linear_search(d, k), -1)
            self.assertEqual(binary_search_recursive(s, k), -1)
            self.assertEqual(h.search(k), -1)

    # --- hash insert (daily update) ----------------------------------------
    def test_hash_insert(self):
        h = HashIndex([1, 2, 3])
        h.insert(42)
        self.assertEqual(h.records[h.search(42)], 42)
        self.assertEqual(len(h.records), 4)


if __name__ == "__main__":
    unittest.main(verbosity=2)

```

src tests 2

```
"""
run_experiments.py - Tasks 8 & 9: benchmark the three strategies on 1K / 10K / 100K / 1M records.
Writes results to ../../Task_9_Experimental_Performance/results.json and results.csv.
Usage:  python run_experiments.py            (full run, ~2-3 minutes)
        python run_experiments.py --quick    (sizes up to 100,000 only)
"""
import csv, json, math, os, statistics, sys, time, tracemalloc, platform
from algorithms import (linear_search, linear_search_counted, binary_search_recursive,
                        binary_search_counted, merge_sort, HashIndex)
from data_gen import SIZES, make_dataset, make_queries

OUT = os.path.join(os.path.dirname(__file__), "..", "..", "Task_9_Experimental_Performance")
os.makedirs(OUT, exist_ok=True)


def time_per_search(fn, keys, repeats):
    """Median over `repeats` of (total time / number of keys), in microseconds."""
    runs = []
    for _ in range(repeats):
        t0 = time.perf_counter()
        for k in keys:
            fn(k)
        runs.append((time.perf_counter() - t0) / len(keys) * 1e6)
    return statistics.median(runs)


def print_summary(results):
    line = "-" * 96
    print("\n" + line)
    print("MEAN TIME PER SUCCESSFUL SEARCH (microseconds)  /  MEAN COMPARISONS (successful)")
    print(line)
    print(f"{'n':>10} | {'Linear us':>12} {'Binary us':>10} {'Hash us':>9} | {'Linear cmp':>11} {'Binary cmp':>11} {'Hash cmp
    for r in results:
        print(f"{r['n']:>10,} | {r['linear_us_hit']:>12.2f} {r['binary_us_hit']:>10.3f} {r['hash_us_hit']:>9.3f} | "
              f"{r['linear_cmp_avg_hit']:>11.1f} {r['binary_cmp_avg_hit']:>11.2f} {r['hash_cmp_avg_hit']:>9.3f}")
    print(line)
    print("PREPROCESSING (seconds)  /  WORST-CASE COMPARISONS OBSERVED")
    print(line)
    print(f"{'n':>10} | {'MergeSort s':>12} {'HashBuild s':>12} | {'Linear worst':>12} {'Binary worst':>12} {'Hash worst':>10
    for r in results:
        print(f"{r['n']:>10,} | {r['sort_time_s']:>12.4f} {r['hash_build_s']:>12.4f} | "
              f"{r['linear_cmp_worst']:>12,} {r['binary_cmp_worst']:>12} {r['hash_cmp_worst']:>10}")
    print(line)


def main(sizes):
    results = []
    for n in sizes:
        print(f"\n=== n = {n:,} ===", flush=True)
        data = make_dataset(n)
        q_fast = 2000                       # queries for binary / hash
        q_slow = 100 if n >= 100_000 else 300   # queries for linear
        hits_f, miss_f = make_queries(data, q_fast)
        hits_s, miss_s = hits_f[:q_slow], miss_f[:q_slow]
        r = {"n": n, "machine": platform.processor() or platform.machine(),
```

src experiments 1

```
             "python": platform.python_version()}

        # ---------------- preprocessing ----------------
        reps = 3 if n < 1_000_000 else 1
        ts = []
        for _ in range(reps):
            t0 = time.perf_counter(); sdata, sort_comps = merge_sort(data); ts.append(time.perf_counter() - t0)
        r["sort_time_s"] = statistics.median(ts)
        r["sort_comparisons"] = sort_comps
        r["sort_nlogn_bound"] = n * math.log2(n)
        t0 = time.perf_counter(); _ = sorted(data); r["timsort_builtin_s"] = time.perf_counter() - t0
        assert sdata == _

        ts = []
        for _ in range(reps):
            t0 = time.perf_counter(); H = HashIndex(data); ts.append(time.perf_counter() - t0)
        r["hash_build_s"] = statistics.median(ts)
        r["hash_stats"] = H.chain_stats()
        tracemalloc.start(); _H = HashIndex(data); cur, peak = tracemalloc.get_traced_memory(); tracemalloc.stop()
        r["hash_extra_mem_MB"] = cur / 1e6
        r["sorted_extra_mem_MB"] = (8 * n) / 1e6      # one pointer per record in the sorted copy
        del _H

        # ---------------- timing ----------------
        rep_slow = 3
        r["linear_us_hit"] = time_per_search(lambda k: linear_search(data, k), hits_s, rep_slow)
        r["linear_us_miss"] = time_per_search(lambda k: linear_search(data, k), miss_s, rep_slow)
        r["binary_us_hit"] = time_per_search(lambda k: binary_search_recursive(sdata, k), hits_f, 5)
        r["binary_us_miss"] = time_per_search(lambda k: binary_search_recursive(sdata, k), miss_f, 5)
        r["hash_us_hit"] = time_per_search(H.search, hits_f, 5)
        r["hash_us_miss"] = time_per_search(H.search, miss_f, 5)

        # ---------------- comparison counts ----------------
        lc_h = [linear_search_counted(data, k)[1] for k in hits_f[:1000]]   # 1000 successful queries for a stable mean
        lc_m = [linear_search_counted(data, k)[1] for k in miss_s]
        r["linear_cmp_best"] = linear_search_counted(data, data[0])[1]
        r["linear_cmp_avg_hit"] = statistics.mean(lc_h)
        r["linear_cmp_worst"] = max(lc_m + [linear_search_counted(data, data[-1])[1]])
        bc_h = [binary_search_counted(sdata, k)[1] for k in hits_f]
        bc_m = [binary_search_counted(sdata, k)[1] for k in miss_f]
        r["binary_cmp_best"] = binary_search_counted(sdata, sdata[(n - 1) // 2])[1]
        r["binary_cmp_avg_hit"] = statistics.mean(bc_h)
        r["binary_cmp_avg_miss"] = statistics.mean(bc_m)
        r["binary_cmp_worst"] = max(bc_h + bc_m)
        hc_h = [H.search_counted(k)[1] for k in hits_f]
        hc_m = [H.search_counted(k)[1] for k in miss_f]
        r["hash_cmp_avg_hit"] = statistics.mean(hc_h)
        r["hash_cmp_avg_miss"] = statistics.mean(hc_m)
        r["hash_cmp_worst"] = max(hc_h + hc_m)

        # theory for the same n
        a = n / H.m
        r["theory"] = {
            "linear_avg_hit": (n + 1) / 2, "linear_worst": n,
            "binary_worst": math.floor(math.log2(n)) + 1,
```

src experiments 2

```
            "binary_avg_hit": ((n + 1) / n) * math.log2(n + 1) - 1,
            "hash_avg_hit": 1 + a / 2, "hash_avg_miss": a,
        }
        results.append(r)
        print(f"  done: linear {r['linear_us_hit']:.1f} us | binary {r['binary_us_hit']:.2f} us | hash {r['hash_us_hit']:.3f}

    with open(os.path.join(OUT, "results.json"), "w") as f:
        json.dump(results, f, indent=2)
    flat = [{k: v for k, v in r.items() if not isinstance(v, dict)} for r in results]
    with open(os.path.join(OUT, "results.csv"), "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(flat[0].keys())); w.writeheader(); w.writerows(flat)
    print_summary(results)
    print("\nSaved results.json / results.csv")


if __name__ == "__main__":
    main([s for s in SIZES if s <= 100_000] if "--quick" in sys.argv else SIZES)

```

src experiments 3